บทที่ 6

ฟังก์ชันในภาษาซี

Function in C Language

ทำไมถึงใช้ฟังก์ชั่น ?

ถ้าเราต้องการเขียนโปรแกรมเพื่อหาค่าผลรวมจาก 1 ถึง 10 และจาก 20 ถึง 37

การเขียนคำสั่งโดย**ไม่มี**การเรียกใช้ฟังก์ชัน

```
int sum = 0
for (int i = 1; i <=10; i++)
        sum +=i;
printf("Sum from 1 to 10 is %d",sum);
sum = 0
for (int i = 20; i <=37; i++)
        sum +=i;
printf("Sum from 20 to 37 is %d",sum);
```

เขียนคำสั่งเดิม **2** ครั้ง

การเขียนคำสั่งโดย**มี**การเรียกใช้ฟังก์ชัน

```
int sum(int i1, int i2){
    int sum = 0;
    for(int i = i1, i <=i2;i++)
        sum += i;
return sum; }

int main(){
  printf("Sum from 1 to 10 is %d\n", sum(1,10));
  printf("Sum from 20 to 37 is %d",sum(20,37));
}
```

ฟังก์ชัน (Function)

- ฟังก์ชันในภาษาซีแบ่งออกเป็น 2 ประเภท คือ
  - ฟังก์ชันมาตรฐานของภาษาซี (Standard Library Functions)
    - ฟังก์ชันที่ภาษาซีจัดเตรียมไว้ให้แล้ว เพียงแต่ต้องทราบก่อนว่าฟังก์ชันนั้นมีการประกาศไว้ในเฮดเดอร์ไฟล์ใด แล้วเพิ่มบรรทัดคำสั่ง

      #include <ชื่อเฮดเดอร์ไฟล์> ← กำหนดไว้ที่หัวโปรแกรม
    - ตัวอย่างเช่น #include <stdio.h>

      ถ้าไม่มีจะไม่สามารถเรียกใช้ scanf() หรือ printf()
  - ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง (User-defined function)
  - **main()** ถือเป็นฟังก์ชันชนิดหนึ่งที่เป็นจุดเริ่มต้นทำงานของโปรแกรม

ฟังก์ชันมาตรฐาน (Standard Function)

Standard Function เป็นฟังก์ชันสำเร็จรูปที่มีมาให้พร้อมกับ Compiler โดยเก็บอยู่ในไฟล์ *.h ซึ่งมีรูปแบบดังนี้ #include <file.h>

Library Function ที่สำคัญ

- math.h ฟังก์ชันการคำนวณทางคณิตศาสตร์
- string.h ฟังก์ชันสำหรับข้อความ
- ctype.h ฟังก์ชันสำหรับอักขระ

ฟังก์ชันการคำนวณทางคณิตศาสตร์ – math.h

Function	Description
sin(x)	sine ของ x, x มีหน่วยเป็นเรเดียน
cos(x)	cosine ของ x, x มีหน่วยเป็นเรเดียน
tan(x)	tangent ของ x, x มีหน่วยเป็นเรเดียน
sqrt(x)	รากที่สองของ x, x>=0
pow(x,y)	x ยกกำลัง y
log(x)	logarithm ฐาน e ของ x, x>=0
log10(x)	logarithm ฐาน 10 ของ x, x>=0
exp(x)	e ยกกำลัง x
abs(x)	ค่าสัมบูรณ์ของ x, x เป็นเลขจำนวนเต็ม
fabs(x)	ค่าสัมบูรณ์ของ x, x เป็นเลขจำนวนจริง

$$\text{Radians} = \left(\frac{\pi}{180^0}\right) \times \text{degrees}$$

$$\text{Degrees} = \left(\frac{180^0}{\pi}\right) \times \text{radians}$$

ฟังก์ชั่นที่ผู้เขียนโปรแกรมสร้างขึ้นเอง มาตรฐานของภาษาซี (User-defined function)

- ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง มาตรฐานของภาษาซี (User-defined function) สามารถแบ่งออกเป็น 4 ประเภท คือ
  - ฟังก์ชันที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน
  - ฟังก์ชันที่**มี**การรับค่าเข้ามาในฟังก์ชัน แต่**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน
  - ฟังก์ชันที่**มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน
  - ฟังก์ชันที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน

โครงสร้างของฟังก์ชันสร้างเอง

1 ประเภทข้อมูลส่งกลับ

2 ชื่อฟังก์ชั่น

3 ประเภทข้อมูลส่งกลับ

4 ชื่อตัวแปรนำเข้า

```
int max2 (int a, int b)
{    if(a > b)
         return a;
     else
         return b;
}
```

ส่วนการประกาศฟังก์ชัน

โครงสร้างของฟังก์ชันสร้างเอง

ส่วนการ
ประกาศฟังก์ชัน

```
int max2(int a, int b)
{    if(a > b)
         return a;
     else
         return b;
}
```

5

return เพื่อส่งค่ากลับ

คำสั่งในฟังก์ชัน

1 ฟังก์ชันที่ไม่มีการรับค่าเข้ามาในฟังก์ชันและการส่งค่ากลับออกไปจากฟังก์ชัน

1 ประเภทข้อมูลส่งกลับ

2 ชื่อฟังก์ชัน

3 ประเภทข้อมูลส่งกลับ

```
void function name(void)
{
        statement;
}
```

1 ประเภทข้อมูลส่งกลับกำหนด void ไม่มีการส่งคืนค่าใดกลับออกไปจากฟังก์ชัน

3 ประเภทข้อมูลส่งกลับกำหนด void ไม่รับค่าใดๆ เข้ามาฟังก์ชัน

1 ฟังก์ชันที่ไม่มีการรับค่าเข้ามาในฟังก์ชันและการส่งค่ากลับออกไปจากฟังก์ชัน

```
1   #include <stdio.h>
2   void do_it(void){
3     printf("Do or do not There is no TRY\n");
4   }
5   void pratice(void){
6     printf("The more you practice, the better you will become\n");
7   }
8   int main(void)
9   {
10    do_it();
11    pratice();
12
13    return 0;
14  }
15
```

Invoke a function

การเรียกใช้งานฟังก์ชัน

ผลลัพธ์

```
Do or do not There is no TRY
The more you practice, the better you will become
```

2 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชันแต่ไม่มีการส่งค่ากลับออกไปจากฟังก์ชัน

1 ประเภทข้อมูลส่งกลับ

2 ชื่อฟังก์ชัน

3 ประเภทข้อมูลส่งกลับ

```
void function name(formal parameters)
{
        statement;
}
```

```
void test_max2(int a, int b)
{
    ...
}
main(){
test_max2(x,y);
}
```

actual parameters

(argument)

Parameters and Arguments

list of parameters

```
void test_max2(int a, int b)
{
    ...
}
main(){
int x, y;
test_max2(x,y);
}
```

actual parameters

(argument)

พารามิเตอร์ (parameter) ตัวรับ โดยฟังก์ชั่นย่อยจะกำหนด พารามิเตอร์เพื่อรองรับค่าพารามิเตอร์จาก main() ในตัวอย่าง มี พารามิเตอร์ 2 ตัว คือ a และ b

อาร์กิวเมนต์ (argument) ตัวส่งโดยฟังก์ชัน main()จะเป็นผู้ส่งพารามิเตอร์ไปฟังก์ชั่นย่อย ในตัวอย่าง มี อาร์กิวเมนต์ 2 ตัวคือ x และ y

⚠ ข้อควรระวัง การกำหนดชนิดข้อมูลของพารามิเตอร์แต่ละตัวจะต้องพิจารณาจากชนิดข้อมูลของอาร์กิวเมนต์ที่จะส่งมาเป็นหลัก main() ส่งค่า x และ y ซึ่งเป็นข้อมูลชนิด int ดังนั้น a และ b จะต้องกำหนดเป็น int เช่นกัน

2 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชันแต่ไม่มีการส่งค่ากลับออกไปจากฟังก์ชัน

```
1   #include <stdio.h>
2   void practices(int n){
3     while(n-- > 0)
4       printf("the better you will become\n");
5   }
6
7   int main(void) {
8     int i;
9     printf("How much do I have to practics: ");
10    scanf("%d",&i);
11    practices(i);
12    return 0;
13  }
```

ผลลัพธ์

```
How much do I have to practice : 3
the better you will become
the better you will become
the better you will become
```

3 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

1 ประเภทข้อมูลส่งกลับ

2 ชื่อฟังก์ชัน

3 ประเภทข้อมูลส่งกลับ

```
return value type function name (formal parameters)
{
    // Function body
return Value; //ค่าที่ต้องการส่งคืนกลับออกไปจากฟังก์ชัน
}
```

```
1 int max2(int a, int b)
{    if(a > b)
  2      return a;
     else
         return b;
}
```

1 และ 2 ชนิดข้อมูลที่ต้องการส่งออกไป ต้องเป็นชนิดข้อมูลเดียวกับ ค่าที่ต้องการส่งคืนกลับออกไปจากฟังก์ชัน

3 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

```
1   #include <stdio.h>
2   int max2(int a, int b){
3     if (a > b)
4       return a;
5     else
6       return b;
7   }
8   int main(void) {
9     int n1,n2;
10    printf("Enter two numbers\n");
11    printf("1st : ");scanf("%d",&n1);
12    printf("2nd : ");scanf("%d",&n2);
13    printf("The bigger number :%d",max2(n1,n2));
14    return 0;
15  }
```

ผลลัพธ์

```
Enter two number:
1st : 45
2nd : 83
The bigger number :83
```

3 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

```
int main(void)
{     /* … */
    printf("The bigger number: \n", max(n1,n2));
    /* … */}
```

45

83

```
int max2(int a, int b)
{    if(a > b)
         return a;
     else
         return b;
}
```

```
int max2(int a, int b)
{    if(a > b)
         return a;
     else
         return b;
}
```

3 ฟังก์ชันที่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

```
int main(void)
{     /* … */
    printf("The bigger number: \n", max(n1,n2) );
    /* … */}
```

45 83

```
int max2(int a, int b)
{    if(a > b)
         return a;
     else
         return b;
}
```

```
a = n1
b = n2
```

ค่าที่ส่งคืนกลับออกไปจากฟังก์ชัน

4 ฟังก์ชันไม่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

```
return value type function name(void)
{
        // Function body
return value; //ค่าที่ต้องการส่งคืนกลับออกไปจากฟังก์ชัน
}
```

```
int max(void)
{
   return max;
}
int min(void){
   return min;
}
```

```
main(){

   int x, y;
   x = max();
   y = min();
}
```

4 ฟังก์ชันไม่มีการรับค่าเข้ามาในฟังก์ชัน และมีการส่งค่ากลับออกไปจากฟังก์ชัน

```
1   #include <stdio.h>
2   int max(void){
3     int n1,n2;
4     printf("Enter two numbers\n");
5     printf("1st : ");scanf("%d",&n1);
6     printf("2nd : ");scanf("%d",&n2);
7     if (n1 > n2)
8       return n1;
9     else
10      return n2;
11  }
12  int main(void) {
13    int x;
14    x = max();
15    printf("The bigger number :%d",x);
16    return 0;
17  }
```

1

2

```
Enter two number:
1st : 45
2nd : 83
The bigger number :83
```

ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง มาตรฐานของภาษาซี (User-defined function)

- **สรุป** ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง มาตรฐานของภาษาซี (User-defined function) สามารถแบ่งออกเป็น **4 ประเภท** คือ
  - ฟังก์ชันที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน → void function name(void)
  - ฟังก์ชั่นที่**มี**การรับค่าเข้ามาในฟังก์ชัน แต่**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน

    → void function name(formal parameters)
  - ฟังก์ชันที่**มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน

    → return value type function name(formal parameters)
  - ฟังก์ชันที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน

    → return value type function name(void)

การประกาศฟังก์ชั่นผู้ใช้สร้างเอง

- รูปแบบของการจัดวางฟังก์ชันย่อยไว้ในโปรแกรม **มี 2 รูปแบบ**
  - ฟังก์ชั่นผู้ใช้สร้างเอง เปรียบเหมือนการสร้างตัวแปร คือ ต้องประกาศก่อนแล้วสามารถเรียกใช้งานได้ ดังนั้น การประกาศฟังก์ชันผู้ใช้สร้างเองนั้นกำหนดไว้หลังส่วนของตัวประมวลผลก่อน (Preprocessor) เสมอ แต่ประกาศฟังก์ชันไว้หลังฟังก์ชัน main() จะต้องมีการประกาศต้นแบบฟังก์ชัน (Function Prototype)

วางฟังก์ชัน main () ไว้ใต้ฟังก์ชันย่อย

```
#include <stdio.h>
void practices(int n){

}
int main(void) {

}
```

วางฟังก์ชัน main () ไว้เหนือฟังก์ชันย่อย

```
#include <stdio.h>
void practices(int n);
int main(void) {

}
void practices(int n){

}
```

⚠️ **ต้องทำการประกาศ function prototype**

การเขียน Prototype ของฟังก์ชัน

type function name(type);

- type ประเภทของข้อมูลที่ส่งกลับ
- function name ชื่อฟังก์ชัน
- type variable ประเภทและชื่อของตัวแปร

```
1   #include <stdio.h>
2
3   void do_it(void);
4   int main(void)
5   {
6     do_it();
7     return 0;
8   }
9   void do_it(void){
10    printf(" Do or do not There is no TRY\n");
11  }
```

⚠ ต้องทำการประกาศ function prototype

ชนิดตัวแปรที่ใช้ในฟังก์ชัน

ชนิดของตัวแปรแยกตามขอบเขตการทำงาน

- **ตัวแปร Global** เป็นตัวแปรที่เรียกใช้จากที่ใดก็ได้ในโปรแกรมโดยจะถูกประกาศไว้หลัง Preprocessor Directive

- **ตัวแปร Local** ถูกสร้างขึ้นภายในฟังก์ชัน การเปลี่ยนแปลงจะมีผลภายในฟังก์ชันเท่านั้น หากมีชื่อซ้ำกันจะถือว่าเป็นคนละตัวแปรกัน ตัวแปรประเภทนี้เมื่อโปรแกรมออกจากฟังก์ชันจะถูกทำลาย

ชนิดตัวแปรที่ใช้ในฟังก์ชัน

Global variable

```
1   #include <stdio.h>
2   int num = 19;
3   void increase()
4   {
5     num = num + 5;
6   }
7   void main()
8   {
9     printf("num = %d\n", num);
10    increase();
11    printf("num = %d\n", num);
12  }
```

ผลลัพธ์

num = 19

num = 24

Local variable

```
1   #include <stdio.h>
2   void increase()
3   { int num = 2;
4     num = num + 5;
5   }
6   void main()
7   { int num = 19;
8     printf("num = %d\n", num);
9     increase();
10    printf("num = %d\n", num);
11  }
12
```

ผลลัพธ์

num = 19

num = 19

ประเภทการรับส่งค่าข้อมูลของฟังก์ชัน

การรับส่งค่าข้อมูลของฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง มาตรฐานของภาษาซี (User-defined function) สามารถแบ่งออกเป็น 3 ประเภท คือ

- **ฟังก์ชันประเภทไม่มีการรับส่งค่าใดๆ**
  - ฟังก์ชั่นที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน
- **ฟังก์ชันแบบส่งค่าผ่านทางเดียว**
  - ฟังก์ชั่นที่**มี**การรับค่าเข้ามาในฟังก์ชัน แต่**ไม่มี**การส่งค่ากลับออกไปจากฟังก์ชัน
  - ฟังก์ชั่นที่**ไม่มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน
- **ฟังก์ชันแบบส่งค่าสองทาง**
  - ฟังก์ชันที่**มี**การรับค่าเข้ามาในฟังก์ชัน และ**มี**การส่งค่ากลับออกไปจากฟังก์ชัน

ประเภทการรับส่งค่าข้อมูลของฟังก์ชัน

การรับค่าข้อมูลจากนอกฟังก์ชันมี 2 แบบ คือ

- pass by value
  - การรับค่าโดยตรง คือ การรับค่าจากตัวแปร ค่าคงที่ หรือ นิพจน์ใดๆ เข้ามาภายในฟังก์ชัน เมื่อมีการประมวลผลเกิดขึ้นกับค่าที่รับเข้ามา จะไม่ส่งผลกระทบกับตัวแปรที่ส่งค่าเข้ามา
- pass by reference
  - การรับค่าด้วยตำแหน่งที่อยู่ของข้อมูล หมายถึง การรับค่าตำแหน่งที่อยู่ของข้อมูล เมื่อเรากระทำการใดๆ กับตัวแปรที่รับค่าเข้ามาให้เกิดการเปลี่ยนแปลงค่าในตัวแปร ค่านั้นจะถูกเปลี่ยนแปลงไปด้วย ถึงแม้ว่าจะเรียกใช้งานตัวแปรนั้นจากนอกฟังก์ชันผู้ใช้สร้างเองที่รับข้อมูลเข้ามาก็ตาม หลักการนี้ในภาษาซีเรียกว่า พอยน์เตอร์ (pointer)

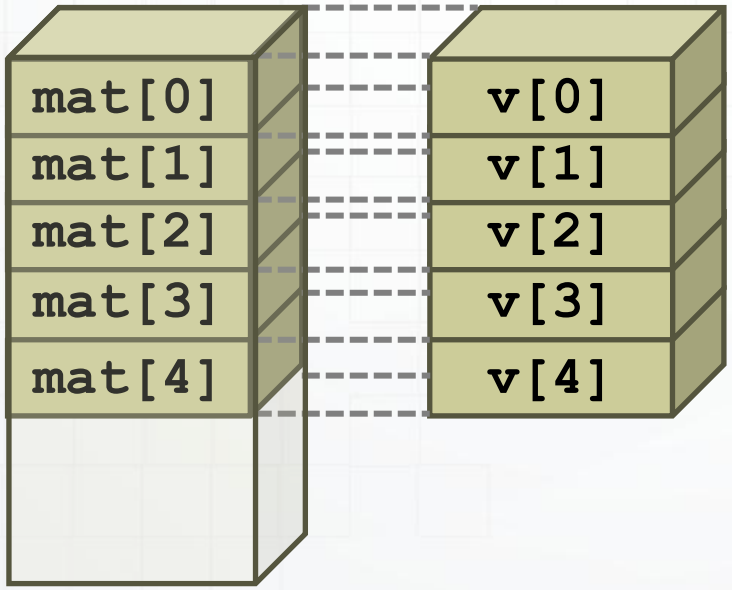
ประเภทการรับส่งค่าข้อมูลของฟังก์ชัน

การรับค่าข้อมูลจากนอกฟังก์ชันมี 2 แบบ คือ

ฟังก์ชั่นของอาเรย์ (Array)

การส่งค่าฟังก์ชันผ่านพารามิเตอร์สามารถส่งในรูปแบบอาร์เรย์ได้ จากข้อดีของอาเรย์เพื่อลดการประกาศพารามิเตอร์จำนานมาก

```
max_of(mat, NUMBER)
```




```
int main(void)
{

   int mat[NUMBER];
   /* . . . */
   max_of(mat, NUMBER)
   /* . . . */

}
```

```
int max_of(int v[], int n)
{
   int i;
   int max = v[0];
   /* . . .*/
   return max
}
```

การรับข้อมูลของฟังก์ชันในรูปแบบอาร์เรย์

```
1   #include <stdio.h>
2   int max_of(int v[],int n);
3   int main(){
4     int mat[5] = {45,36,72,90,100};
5     int max_m;
6     max_m = max_of(mat,5);
7     printf("max value of score %d",max_m);
8
9   }
10  int max_of(int v[],int n) {
11    int max = v[0];
12    for (int i = 1; i< n; i++){
13      if(v[i]> max)
14         max = v[i];}
15    return max;
16  }
```

max value of score 100

จบบทที่ 6

Function